DAIRYCHAIN

Business Questions & SQL Solutions

15 business questions answered with dataset findings, calculation logic, and PostgreSQL queries.

PROJECT DATASET	ANALYSIS SCOPE
10 CSV tables | 99,584 source rows	Procurement, quality, production, sales
Nov 2025 - Oct 2026	Python / Pandas calculations
PostgreSQL query examples	SQL solutions are provided for reproduction

Prepared as a project analysis and learning reference

This report explains the business question first, shows the query, reports the observed answer, and then explains what the result means. Results are calculated from the supplied CSV files with Python and Pandas. The SQL examples are equivalent PostgreSQL solutions and are not represented as executed database output.

Important: these figures describe this project dataset only. They are not external market prices, factory benchmarks, or proof of real-world dairy performance.

How to read this report

Every question follows the same four steps: (1) translate the business question into a measurable metric, (2) identify the required tables and join keys, (3) aggregate or compare the data, and (4) interpret the result without overstating what it proves.

How the answers were calculated

```
import pandas as pd

procurement = pd.read_csv("milk_procurement.csv")
suppliers = pd.read_csv("suppliers.csv")

# Recalculate procurement cost from the business rule:
# litres received x purchase rate per litre
procurement["calculated_cost"] = (
    procurement["quantity_litres"] *
    procurement["purchase_rate_per_litre"]
).round(2)

supplier_summary = (
    procurement.groupby("supplier_id")
    .agg(
        litres=("quantity_litres", "sum"),
        spend=("calculated_cost", "sum")
    )
    .reset_index()
)
```

Python/Pandas was used to read and calculate the results directly from the 10 CSV source files. The SQL below is the database equivalent, written against the project Silver-layer table names (for example, silver.milk_procurement).

Evidence labels

Label	Meaning
Calculated from CSVs	The result was computed from the supplied CSV files using Pandas.
SQL solution	A PostgreSQL query that reproduces the metric. It is a query example, not a claim that this exact statement was executed.
Assumption	A business rule needed to make the metric meaningful; it is stated beside the result.

Tables and join keys used

Subject	Tables	Join key / grain
Supplier procurement	milk_procurement + suppliers	supplier_id; one procurement record per row
Quality	quality_tests + milk_procurement + suppliers	batch_id, then supplier_id
Centre quality	milk_procurement + collection_centres	collection_centre_id
Production	production_batches + production_inputs + production_outputs + products	production_id and product_id
Sales	sales_orders + sales_order_lines + products	order_id and product_id

Metric discipline

A total spend answers “who accounts for the most spend?”; an average rate answers “who is paid more per litre?” Those are different questions. Rejection rate by record count is not the same as rejection rate by milk volume. Production outputs in litres and kilograms must not be compared without a unit conversion. Sales revenue uses the recorded line_total, which matches fulfilled quantity × price less discount.

QUESTION 01

Which suppliers account for the highest milk-purchasing spend?

What this asks: For every supplier, calculate how many litres were purchased and how much the business paid. Then rank suppliers by total spend.

How we worked it out: Group procurement rows by supplier_id, add the quantity and recompute each row cost as quantity_litres × purchase_rate_per_litre. Join suppliers to display names. We use the recalculated amount so the known recorded-cost error does not inflate the ranking.

PostgreSQL solution (reproducible query)

```
SELECT
    s.supplier_id,
    s.supplier_name,
    COUNT(*) AS procurement_records,
    ROUND(SUM(p.quantity_litres), 2) AS milk_litres,
    ROUND(SUM(ROUND(
        p.quantity_litres * p.purchase_rate_per_litre, 2
    )), 2) AS validated_spend_inr
FROM silver.milk_procurement AS p
JOIN silver.suppliers AS s
  ON s.supplier_id = p.supplier_id
GROUP BY s.supplier_id, s.supplier_name
ORDER BY validated_spend_inr DESC
LIMIT 10;
```

Answer from the dataset: Ganesh Jadhav Dairy (SUP-0132) ranks first: 112 procurement records, 57,396.83 litres, and ₹3,175,190.94 (about ₹31.75 lakh) in recalculated spend. It is also the highest-volume supplier in this dataset.

Business interpretation: This supplier contributes the largest total procurement spend mainly because it supplies the largest volume. This does not mean it has the highest purchase rate per litre; Question 2 checks that separately.

Important limitation: Spend is recalculated from quantity × rate. Procurement record PRC-001000 contains a recorded-cost error, so summing the uncorrected total_purchase_cost column would overstate spend by ₹6,657.17.

QUESTION 02

Are different suppliers paid different prices per litre?

What this asks: Compare each supplier’s average purchase rate. Also inspect the lowest and highest transaction rates in the dataset.

How we worked it out: Calculate AVG(purchase_rate_per_litre), MIN and MAX for each supplier. The weighted average uses total recalculated cost divided by total litres; it gives larger purchases more influence than a simple average.

PostgreSQL solution (reproducible query)

```
SELECT
    s.supplier_id,
    s.supplier_name,
    COUNT(*) AS records,
    ROUND(AVG(p.purchase_rate_per_litre), 2) AS avg_rate,
    ROUND(MIN(p.purchase_rate_per_litre), 2) AS min_rate,
    ROUND(MAX(p.purchase_rate_per_litre), 2) AS max_rate
FROM silver.milk_procurement AS p
JOIN silver.suppliers AS s
  ON s.supplier_id = p.supplier_id
GROUP BY s.supplier_id, s.supplier_name
ORDER BY avg_rate DESC;

-- Weighted average price across all procurement
SELECT ROUND(
    SUM(ROUND(quantity_litres * purchase_rate_per_litre, 2)) /
    NULLIF(SUM(quantity_litres), 0), 2
) AS weighted_avg_rate
FROM silver.milk_procurement;
```

Answer from the dataset: Yes. Transaction rates range from ₹47.59 to ₹64.00 per litre. Supplier-level average rates range from ₹54.40 to ₹55.84. Ganesh Reddy Dairy (SUP-0183) has the highest supplier average (₹55.84/L); Suresh Verma Dairy (SUP-0010) has the lowest (₹54.40/L). The overall quantity-weighted rate is about ₹55.02/L.

Business interpretation: The dataset records real variation in the rate field, but the spread between supplier averages is much narrower than the spread between individual transactions. Check contract terms, fat/SNF incentives, timing, and collection conditions before calling a rate difference unfair.

Important limitation: Rate comparisons should ideally control for milk quality, season, supplier agreement, and transaction date. This query is descriptive, not a causal explanation.

QUESTION 03

Which suppliers provide the strongest quality for the price paid?

What this asks: Compare suppliers on their accepted-quality percentage and average purchase rate. We want a high acceptance rate without ignoring price.

How we worked it out: Join each quality test to its procurement batch, then to the supplier. Rank by acceptance percentage. To avoid ranking a supplier on only a handful of records, keep suppliers with at least 30 tests. Review price alongside quality rather than hiding the trade-off inside an arbitrary score.

PostgreSQL solution (reproducible query)

```
SELECT
    s.supplier_id,
    s.supplier_name,
    COUNT(*) AS tests,
    COUNT(*) FILTER (
        WHERE q.quality_status = 'Accepted'
    ) AS accepted_tests,
    COUNT(*) FILTER (
        WHERE q.quality_status = 'Rejected'
    ) AS rejected_tests,
    ROUND(100.0 * COUNT(*) FILTER (
        WHERE q.quality_status = 'Accepted'
    ) / NULLIF(COUNT(*), 0), 2) AS acceptance_pct,
    ROUND(AVG(p.purchase_rate_per_litre), 2) AS avg_rate
FROM silver.quality_tests AS q
JOIN silver.milk_procurement AS p
  ON p.batch_id = q.batch_id
JOIN silver.suppliers AS s
  ON s.supplier_id = p.supplier_id
GROUP BY s.supplier_id, s.supplier_name
HAVING COUNT(*) >= 30
ORDER BY acceptance_pct DESC, avg_rate ASC
LIMIT 10;
```

Answer from the dataset: Nitin Jadhav Dairy (SUP-0131) has the highest acceptance rate among suppliers with at least 30 tests: 88 accepted out of 91 tests (96.70%), at an average rate of ₹55.26/L. For comparison, the lowest-average-rate supplier, Suresh Verma Dairy, averages ₹54.40/L but has an 89.02% acceptance rate (73 of 82 tests).

Business interpretation: The top-quality supplier costs about ₹0.86/L more than the lowest-average-rate supplier, but its observed acceptance rate is about 7.68 percentage points higher. This is a useful supplier-review comparison, not proof that paying more causes better quality.

Important limitation: “Best value” depends on how the business values accepted milk, rejected milk, testing costs, and supplier reliability. This report uses acceptance percentage as the primary quality measure and displays average rate beside it.

QUESTION 04

Are recorded milk-purchase costs calculated correctly?

What this asks: Check whether each procurement cost equals litres received multiplied by the purchase rate per litre.

How we worked it out: Create an expected cost for each row, round it to two decimal places, subtract it from the recorded amount, and flag absolute differences above ₹0.01.

PostgreSQL solution (reproducible query)

```
SELECT
    procurement_id,
    quantity_litres,
    purchase_rate_per_litre,
    total_purchase_cost AS recorded_cost,
    ROUND(quantity_litres * purchase_rate_per_litre, 2)
        AS calculated_cost,
    ROUND(
        total_purchase_cost -
        ROUND(quantity_litres * purchase_rate_per_litre, 2), 2
    ) AS difference_inr
FROM silver.milk_procurement
WHERE ABS(
    total_purchase_cost -
    ROUND(quantity_litres * purchase_rate_per_litre, 2)
) > 0.01
ORDER BY ABS(
    total_purchase_cost -
    ROUND(quantity_litres * purchase_rate_per_litre, 2)
) DESC;
```

Answer from the dataset: One mismatch was found: PRC-001000. Quantity = 510.03 L; rate = ₹52.21/L; calculated cost = ₹26,628.67; recorded cost = ₹33,285.84; overstatement = ₹6,657.17. The other 17,999 records match within the one-paise tolerance.

Business interpretation: Do not silently overwrite the source cost. Preserve the original value, store the calculated value, and flag the exception for review. The row belongs to supplier Rajesh Rathod Dairy (SUP-0155).

Important limitation: The result is a data-quality exception. A finance or procurement owner should confirm which value is authoritative before correcting the official record.

QUESTION 05

How much could a 2% reduction in milk-purchasing costs save?

What this asks: Apply a 2% reduction to the current validated procurement spend to estimate the size of the opportunity.

How we worked it out: First recalculate spend from quantity × rate, then multiply the total by 0.02. This is a scenario calculation, not a forecast or proof that a 2% reduction is achievable.

PostgreSQL solution (reproducible query)

```
WITH validated_spend AS (
    SELECT SUM(ROUND(
        quantity_litres * purchase_rate_per_litre, 2
    )) AS spend_inr
    FROM silver.milk_procurement
)
SELECT
    ROUND(spend_inr, 2) AS baseline_spend_inr,
    ROUND(spend_inr * 0.02, 2) AS potential_2pct_saving_inr
FROM validated_spend;
```

Answer from the dataset: Validated spend across all procurement records is ₹481,919,772.07 (about ₹48.19 crore). A 2% reduction would represent approximately ₹9,638,395.44 (about ₹96.38 lakh) over the dataset period.

Business interpretation: This gives management a scale for prioritising rate negotiations, quality improvement, and supplier consolidation discussions. It is not a guaranteed saving.

Important limitation: The main figure includes rejected procurement records because the dataset records a cost for them. If rejected milk is not paid for under the real commercial policy, the accepted-only baseline is ₹422,004,105.05 and 2% of that is ₹8,440,082.10. The reporting rule must be confirmed.

QUESTION 06

Why is milk being rejected?

What this asks: Count the reasons attached to rejected quality tests and calculate each reason's share of all rejected tests.

How we worked it out: Filter quality_tests to Rejected, group by rejection_reason, count tests, and divide each count by the rejected-test total.

PostgreSQL solution (reproducible query)

```
SELECT
    rejection_reason,
    COUNT(*) AS rejected_tests,
    ROUND(100.0 * COUNT(*) /
        SUM(COUNT(*)) OVER (), 2) AS share_of_rejections_pct
FROM silver.quality_tests
WHERE quality_status = 'Rejected'
GROUP BY rejection_reason
ORDER BY rejected_tests DESC;
```

Answer from the dataset: There are 2,311 rejected tests (12.84% of 18,000 tests). Reasons: Low SNF = 1,303 (56.38%); Low Fat = 513 (22.20%); High Acidity = 491 (21.25%); High Temperature = 4 (0.17%).

Business interpretation: Low SNF is the largest recorded reason and should be the first investigation priority. Review supplier-level SNF patterns, sampling, storage, and test procedures before deciding on corrective action.

Important limitation: These percentages are based on rejection records, not rejected litres. A volume-weighted reason analysis could produce a different ranking.

QUESTION 07

Which suppliers repeatedly provide lower-quality milk?

What this asks: Identify suppliers with many rejected tests and a high rejection percentage, rather than reacting to one isolated failed test.

How we worked it out: Join quality tests to procurement by batch_id, group by supplier, count tests and rejections, calculate the rejection rate, and restrict the ranking to suppliers with at least 30 tests.

PostgreSQL solution (reproducible query)

```
SELECT
    s.supplier_id,
    s.supplier_name,
    COUNT(*) AS tests,
    COUNT(*) FILTER (
        WHERE q.quality_status = 'Rejected'
    ) AS rejected_tests,
    ROUND(100.0 * COUNT(*) FILTER (
        WHERE q.quality_status = 'Rejected'
    ) / NULLIF(COUNT(*), 0), 2) AS rejection_pct
FROM silver.quality_tests AS q
JOIN silver.milk_procurement AS p
  ON p.batch_id = q.batch_id
JOIN silver.suppliers AS s
  ON s.supplier_id = p.supplier_id
GROUP BY s.supplier_id, s.supplier_name
HAVING COUNT(*) >= 30
ORDER BY rejection_pct DESC, rejected_tests DESC
LIMIT 10;
```

Answer from the dataset: Dinesh Pawar Dairy (SUP-0087) ranks highest among suppliers with at least 30 tests: 21 rejected out of 84 (25.00%). Next are Rajesh Jadhav Dairy (SUP-0024), 22/93 (23.66%); Ganesh Reddy Dairy (SUP-0038), 18/83 (21.69%); Anil Verma Dairy (SUP-0003), 17/81 (20.99%); and Vikram Singh Dairy (SUP-0083), 20/97 (20.62%).

Business interpretation: These suppliers are sensible candidates for a focused quality review, including SNF/fat trends, collection handling, and repeat testing. Review the cause before changing supplier terms.

Important limitation: The ranking is based on test counts rather than volume-weighted rejection. The 30-test threshold is a screening choice to reduce small-sample noise, not a universal standard.

QUESTION 08

Do suppliers who deliver more milk also provide better-quality milk?

What this asks: Check whether suppliers with higher total litres tend to have a higher acceptance percentage.

How we worked it out: For each supplier, calculate total litres and accepted tests divided by total tests. Then calculate the Pearson correlation between those two supplier-level measures.

PostgreSQL solution (reproducible query)

```
WITH supplier_metrics AS (
    SELECT
        p.supplier_id,
        SUM(p.quantity_litres) AS total_litres,
        100.0 * COUNT(*) FILTER (
            WHERE q.quality_status = 'Accepted'
        ) / NULLIF(COUNT(*), 0) AS acceptance_pct
    FROM silver.milk_procurement AS p
    JOIN silver.quality_tests AS q
      ON q.batch_id = p.batch_id
    GROUP BY p.supplier_id
)
SELECT
    COUNT(*) AS suppliers,
    ROUND(CORR(total_litres, acceptance_pct)::numeric, 4)
        AS pearson_correlation
FROM supplier_metrics;
```

Answer from the dataset: Across 200 suppliers, the Pearson correlation is approximately -0.0608.

Business interpretation: This is an extremely weak negative linear relationship. In this dataset, supplying more litres is not meaningfully associated with a higher acceptance percentage. Supplier volume should not be used as a substitute for quality monitoring.

Important limitation: Correlation does not establish cause and can hide differences in season, supplier type, centre, or the number of tests per supplier.

QUESTION 09

Which collection centre rejects the largest share of the milk it receives?

What this asks: Compare rejected litres with total received litres at each collection centre. This is a volume-based rejection rate.

How we worked it out: Join procurement to collection-centre master data, sum all litres, sum rejected litres only, and divide rejected litres by received litres.

PostgreSQL solution (reproducible query)

```
SELECT
    c.collection_centre_id,
    c.centre_name,
    COUNT(*) AS procurement_records,
    ROUND(SUM(p.quantity_litres), 2) AS received_litres,
    ROUND(SUM(CASE WHEN p.acceptance_status = 'Rejected'
                   THEN p.quantity_litres ELSE 0 END), 2)
        AS rejected_litres,
    ROUND(100.0 * SUM(CASE WHEN p.acceptance_status = 'Rejected'
                           THEN p.quantity_litres ELSE 0 END) /
          NULLIF(SUM(p.quantity_litres), 0), 2)
        AS rejection_by_volume_pct
FROM silver.milk_procurement AS p
JOIN silver.collection_centres AS c
  ON c.collection_centre_id = p.collection_centre_id
GROUP BY c.collection_centre_id, c.centre_name
ORDER BY rejection_by_volume_pct DESC;
```

Answer from the dataset: Boriavi Milk Collection Centre (CC-002) is highest: 60,816.91 rejected litres out of 442,288.44 received, a 13.75% volume rejection rate. Umreth is next at 13.60%. Bakrol is lowest at 10.98%.

Business interpretation: Boriavi is the first centre to investigate for milk handling, chilling, time-to-testing, and supplier mix. Compare its processes with lower-rejection centres before deciding what to change.

Important limitation: This is a volume-weighted rate. The overall volume rejection rate is 12.76%, while the record-count rejection rate is 12.84%; these are different metrics.

QUESTION 10

How much more milk could be accepted if rejections fell by 10%?

What this asks: Estimate the extra litres that could be accepted if the current rejected volume were reduced by 10% relative to its present level.

How we worked it out: Sum quantity_litres where acceptance_status is Rejected and multiply that amount by 10%. This is a relative 10% reduction in rejected litres, not a 10-percentage-point reduction in the rejection rate.

PostgreSQL solution (reproducible query)

```
WITH rejected_volume AS (
    SELECT SUM(quantity_litres) AS rejected_litres
    FROM silver.milk_procurement
    WHERE acceptance_status = 'Rejected'
)
SELECT
    ROUND(rejected_litres, 2) AS current_rejected_litres,
    ROUND(rejected_litres * 0.10, 2)
        AS potential_additional_accepted_litres
FROM rejected_volume;
```

Answer from the dataset: Current rejected volume is 1,117,247.35 litres. A 10% relative reduction would represent 111,724.74 litres of additional potentially accepted milk.

Business interpretation: This gives a practical improvement target for the quality programme. Prioritising the dominant low-SNF reason and the highest-rejection suppliers/centres may help focus the investigation.

Important limitation: The calculation assumes the recovered volume becomes acceptable and usable. It does not automatically translate into revenue or finished product; capacity, product mix, and commercial acceptance rules also matter.

QUESTION 11

Are production batches producing a consistent amount of finished product?

What this asks: For each product, compare raw milk input with primary finished-product output and calculate yield using matching units.

How we worked it out: Sum all raw-milk input rows per production_id, then join to the primary-product output and product unit. For kilogram products, convert 650 litres to 669.5 kg using the dataset conversion factor of 1.03 kg/L before calculating the yield ratio.

PostgreSQL solution (reproducible query)

```
WITH input_totals AS (
    SELECT production_id, SUM(input_quantity) AS input_litres
    FROM silver.production_inputs
    GROUP BY production_id
),
primary_outputs AS (
    SELECT production_id, product_id,
           SUM(output_quantity) AS primary_output
    FROM silver.production_outputs
    WHERE output_classification = 'Primary product'
    GROUP BY production_id, product_id
),
batch_yield AS (
    SELECT
        b.production_id, b.product_id, pr.product_name,
        pr.unit_of_measure, i.input_litres, o.primary_output,
        CASE WHEN pr.unit_of_measure = 'Kilogram'
             THEN i.input_litres * 1.03
             ELSE i.input_litres END AS comparable_input
    FROM silver.production_batches AS b
    JOIN input_totals AS i ON i.production_id = b.production_id
    JOIN primary_outputs AS o
      ON o.production_id = b.production_id
     AND o.product_id = b.product_id
    JOIN silver.products AS pr ON pr.product_id = b.product_id
)
SELECT
    product_name, unit_of_measure,
```

```
    COUNT(*) AS batches,
    ROUND(AVG(input_litres), 2) AS avg_input_litres,
    ROUND(AVG(primary_output), 2) AS avg_primary_output,
    ROUND(100.0 * AVG(primary_output / comparable_input), 2)
        AS yield_pct
FROM batch_yield
GROUP BY product_name, unit_of_measure
ORDER BY yield_pct DESC;
```

Answer from the dataset: All 4,000 production batches are marked Completed. Each batch uses 650.00 litres of raw milk. Primary output is consistent within each product: liquid milk products produce 633.75 L per batch (97.50%); flavoured milk 630.50 L (97.00%); buttermilk/lassi 611.00 L (94.00%); paneer 120.51 kg (18.00% of the 669.5 kg input-mass basis); ghee 29.25 L (4.50%); cream 32.50 L (5.00%); butter 30.13 kg (about 4.50%); curd 649.41 kg (about 97.00%).

Business interpretation: The batch outputs follow a consistent product-specific conversion pattern in the supplied data. Use the yield table for comparisons within the same product and unit, not to compare litres directly with kilograms.

Important limitation: Only primary product output is used in the yield metric; the by-product/residual output is kept separate. The 1.03 kg/L conversion is a dataset modelling assumption, not a universal dairy-process constant.

QUESTION 12

Are any production batches unusually low- or high-yield compared with the same product?

What this asks: Look for a production batch whose yield is far from other batches producing the same product.

How we worked it out: Calculate a per-batch yield after converting input to the product output unit. Compare each batch only with batches of the same product; a three-standard-deviation check is a common screening method. Round yield to four decimals to avoid treating floating-point noise as a real process difference.

PostgreSQL solution (reproducible query)

```
WITH input_totals AS (
    SELECT production_id, SUM(input_quantity) AS input_litres
    FROM silver.production_inputs
    GROUP BY production_id
),
primary_outputs AS (
    SELECT production_id, product_id,
           SUM(output_quantity) AS primary_output
    FROM silver.production_outputs
    WHERE output_classification = 'Primary product'
    GROUP BY production_id, product_id
),
batch_yield AS (
    SELECT b.production_id, b.product_id, pr.product_name,
           ROUND(o.primary_output /
               CASE WHEN pr.unit_of_measure = 'Kilogram'
                    THEN i.input_litres * 1.03
                    ELSE i.input_litres END, 4) AS yield_ratio
    FROM silver.production_batches AS b
    JOIN input_totals AS i ON i.production_id = b.production_id
    JOIN primary_outputs AS o
      ON o.production_id = b.production_id
     AND o.product_id = b.product_id
    JOIN silver.products AS pr ON pr.product_id = b.product_id
),
scored AS (
    SELECT *,
        AVG(yield_ratio) OVER (PARTITION BY product_id) AS avg_yield,
        STDDEV_POP(yield_ratio) OVER (PARTITION BY product_id) AS sd_yield
    FROM batch_yield
)
SELECT production_id, product_name, yield_ratio,
       ROUND((yield_ratio - avg_yield) /
             NULLIF(sd_yield, 0), 2) AS z_score
FROM scored
WHERE sd_yield > 0
  AND ABS((yield_ratio - avg_yield) / NULLIF(sd_yield, 0)) > 3
ORDER BY ABS((yield_ratio - avg_yield) / NULLIF(sd_yield, 0)) DESC;
```

Answer from the dataset: No production-yield outlier was identified after comparing like products with the correct unit conversion and rounding. Within each product, the observed primary yield is the same across its batches to four decimal places.

Business interpretation: There is no batch-level yield variation in this dataset to investigate. This is a limitation of the available data: it cannot support realistic process-variation or anomaly-monitoring conclusions.

Important limitation: A first unadjusted comparison of kilograms against litres can create false anomalies. Always align units and compare a batch only with the same product.

QUESTION 13

Which products require more raw milk per production batch?

What this asks: Check the raw milk used by each product per batch, rather than comparing only the total milk used across all batches.

How we worked it out: Aggregate all production_input rows by production_id, join the batch to product master, and average input litres by product. Production inputs can have multiple rows because one run can draw milk from multiple source batches.

PostgreSQL solution (reproducible query)

```
WITH input_by_batch AS (
    SELECT production_id, SUM(input_quantity) AS raw_milk_litres
    FROM silver.production_inputs
    WHERE input_material_type = 'Raw Milk'
    GROUP BY production_id
)
SELECT
    pr.product_name,
    COUNT(*) AS production_batches,
    ROUND(AVG(i.raw_milk_litres), 2) AS avg_raw_milk_litres_per_batch,
    ROUND(SUM(i.raw_milk_litres), 2) AS total_raw_milk_litres
FROM input_by_batch AS i
JOIN silver.production_batches AS b
  ON b.production_id = i.production_id
JOIN silver.products AS pr
  ON pr.product_id = b.product_id
GROUP BY pr.product_id, pr.product_name
ORDER BY avg_raw_milk_litres_per_batch DESC;
```

Answer from the dataset: No product uses more raw milk per batch in this dataset: every one of the 4,000 completed batches consumes 650.00 litres. Total input differs only because some products have 334 batches and others 333. Overall raw-milk input is 2,600,000 litres.

Business interpretation: This question reveals that the model applies a fixed 650-litre input to each batch. To compare recipe intensity in a real factory, input quantities for all ingredients and actual batch outputs would be needed.

Important limitation: Do not rank products by total input alone: a product with one additional batch will have a larger total even though its per-batch requirement is identical.

QUESTION 14

Which products generate the most sales revenue?

What this asks: Add the net sales value across all order lines for each product and rank products from highest to lowest.

How we worked it out: Join order lines to the product master and sum line_total. The line_total in this dataset reconciles to quantity_fulfilled × unit_selling_price − discount_amount, so it represents the net value of fulfilled units.

PostgreSQL solution (reproducible query)

```
SELECT
    pr.product_id,
    pr.product_name,
    COUNT(*) AS order_lines,
    ROUND(SUM(l.quantity_fulfilled), 2) AS units_fulfilled,
    ROUND(SUM(l.line_total), 2) AS net_sales_revenue_inr
FROM silver.sales_order_lines AS l
JOIN silver.products AS pr
```

```
  ON pr.product_id = l.product_id
GROUP BY pr.product_id, pr.product_name
ORDER BY net_sales_revenue_inr DESC;
```

Answer from the dataset: Total net sales revenue is ₹26,734,294.03. The top products are Pure Cow & Buffalo Ghee (₹5,986,642.50; 22.39% of total), Fresh Malai Paneer (₹5,531,071.00), and Pasteurised Table Butter (₹5,156,439.08).

Business interpretation: These products contribute the largest recorded sales value and may deserve attention in pricing, demand planning, and availability reviews. Revenue does not equal profit.

Important limitation: The dataset has no complete product-level cost-of-goods-sold or margin calculation in these tables. High revenue alone does not prove high profitability.

QUESTION 15

How many customer orders were not fully fulfilled or were delivered late?

What this asks: Count cancelled and partially delivered orders, then separately compare actual and expected delivery dates for orders with an actual delivery date.

How we worked it out: First summarise ordered and fulfilled quantities by order_id. An order is fully fulfilled when fulfilled quantity equals ordered quantity. Then inspect the order status and compare actual_delivery_date with expected_delivery_date; cancelled orders have no actual delivery date.

PostgreSQL solution (reproducible query)

```
WITH order_lines AS (
    SELECT order_id,
           SUM(quantity_ordered) AS ordered_qty,
           SUM(quantity_fulfilled) AS fulfilled_qty
    FROM silver.sales_order_lines
    GROUP BY order_id
)
SELECT
    o.order_status,
    COUNT(*) AS orders,
    COUNT(*) FILTER (
        WHERE l.fulfilled_qty < l.ordered_qty
    ) AS not_fully_fulfilled,
    COUNT(*) FILTER (
        WHERE o.actual_delivery_date IS NOT NULL
          AND o.actual_delivery_date > o.expected_delivery_date
    ) AS late_orders
FROM silver.sales_orders AS o
JOIN order_lines AS l ON l.order_id = o.order_id
GROUP BY o.order_status
ORDER BY o.order_status;
```

Answer from the dataset: There are 2,834 orders not fully fulfilled: 2,227 Partially Delivered and 607 Cancelled. The remaining 11,166 are Delivered and fully fulfilled. Among 13,393 non-cancelled orders with an actual delivery date, 0 are later than the expected date; all are recorded on the expected date.

Business interpretation: The primary issue is fulfilment completeness, not recorded delivery lateness. Investigate the 2,227 partially delivered orders separately from the 607 cancelled orders because their operational causes may differ.

Important limitation: “On time” is measured only from the date fields provided. The date fields do not reveal delivery time-of-day, partial quantities at each drop, or the reason an order was cancelled.

Consolidated findings

Area	Finding	What it suggests
Procurement	Top spend supplier: Ganesh Jadhav Dairy; ₹31.75 lakh recalculated spend.	Large spend is primarily associated with high supplied volume.
Purchase rates	Supplier average range: ₹54.40-₹55.84/L; weighted overall rate ₹55.02/L.	Review pricing with quality and contract context.
Supplier quality	Highest acceptance among suppliers with 30+ tests: 96.70% for Nitin Jadhav Dairy.	Use both acceptance rate and rate/L when reviewing value.
Cost control	One cost mismatch: PRC-001000; overstatement ₹6,657.17.	Preserve source value and flag for finance review.
Savings scenario	2% of validated total spend = about ₹96.38 lakh.	Scenario only; confirm whether rejected milk is paid for.
Quality	2,311 rejected tests; Low SNF is 56.38% of rejections.	Low SNF is the first investigation priority.
Collection centres	Boriavi has the highest rejected-volume rate: 13.75%.	Compare centre handling and supplier mix.
Production	Every completed batch uses 650 L; no within-product yield outliers.	This dataset has fixed product recipes and limited process variation.
Sales	Ghee has the highest sales value: ₹59.87 lakh.	Revenue ranking is not a profit ranking.
Order fulfilment	2,834 orders not fully fulfilled; 0 late dates among non-cancelled orders.	Focus on partial fulfilment and cancellations.

Recommended order for a business review

1. Investigate Low SNF rejections and the suppliers/centres with the highest rejection percentages.
2. Review the one procurement-cost exception before reporting procurement spend.
3. Compare supplier price with acceptance percentage instead of choosing suppliers by price alone.
4. Investigate partial fulfilment and cancellations by product and customer code.
5. Treat production yield as a data-model consistency result, not a plant-efficiency benchmark.

How to reproduce the results in PostgreSQL

Connect to the dairychain database, confirm the Silver table names and column types, then execute the relevant query. If the schema is not named silver or the column names differ, adjust the schema/table/column references to match the live database catalogue. Compare the output with the Python-derived reference values in this document.

Data-handling note

Do not overwrite original records simply because a calculation identifies an exception. Keep the original field, create a calculated field, record the rule and flag the mismatch for review. Likewise, do not claim SQL execution just because a query is included in project documentation: attach the actual psql output or database result when execution has been completed.